

AniBaba – AI Anime Recommender System

1. Project Title

AniBaba: AI-Powered Anime Recommendation System using RAG



2. Duration

4–5 Hours

3. Introduction

AniBaba is an AI-powered anime recommendation system that enables users to discover anime through natural language conversations. Instead of relying on traditional genre filters, users can describe moods, story depth, or specific narrative elements they enjoy. The system uses Retrieval-Augmented Generation (RAG) to retrieve semantically relevant anime and generate intelligent, personalized recommendations through a conversational interface.

4. Aim

The aim of this project is to design and implement a conversational anime recommender system that understands user preferences from natural language queries and provides accurate, context-aware anime recommendations using semantic search and large language models also with learning to design deployable softwares.

5. Dataset Used

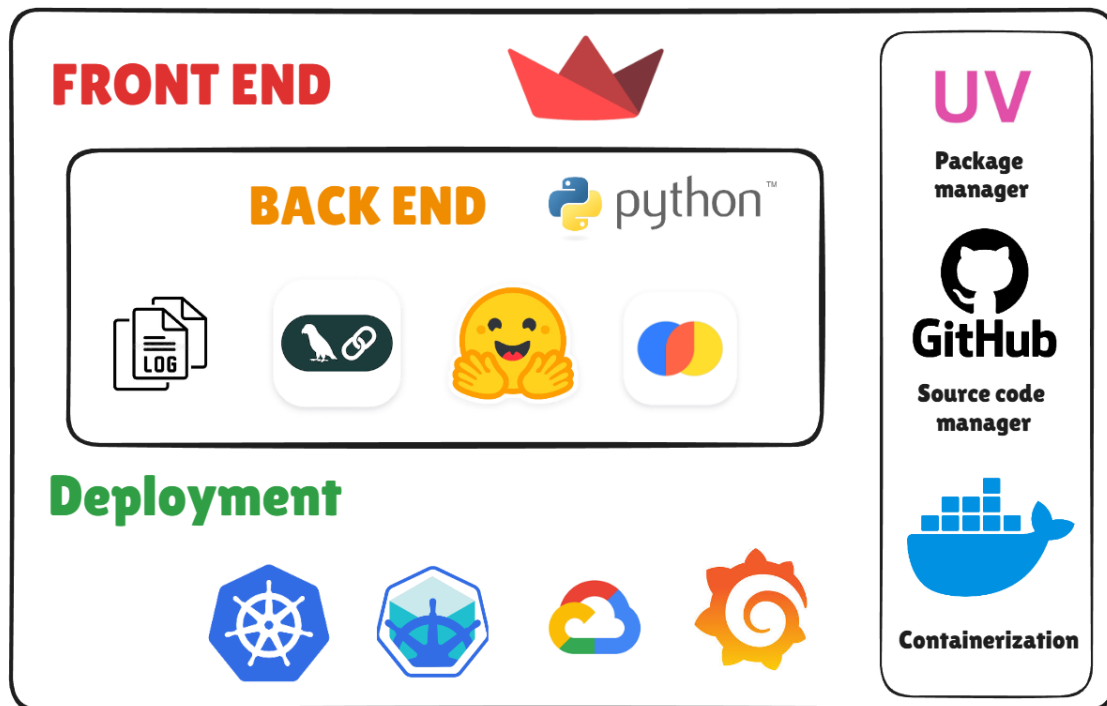
Dataset Name: Anime Data

Format: CSV

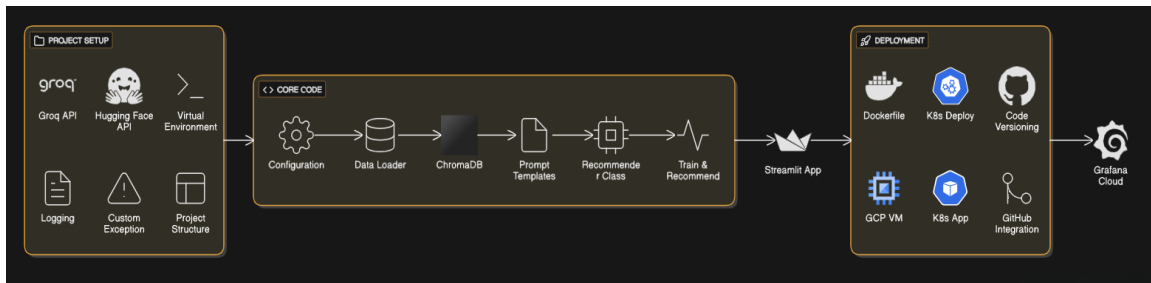
Description: The dataset contains structured anime metadata such as titles, genres, synopsis, themes, and descriptive attributes. This information is used to create semantic embeddings for accurate retrieval during recommendation.

6. Tools & Technologies

- Programming Language: Python
- Frontend: Streamlit
- LLM Orchestration: LangChain
- Embeddings: HuggingFace (all-MiniLM-L6-v2)
- Vector Database: ChromaDB
- Cloud Platform: Google Cloud Platform (GCP)
- Deployment: Docker, Kubernetes (Minikube)
- Monitoring: Grafana Cloud



7. Architecture Diagram



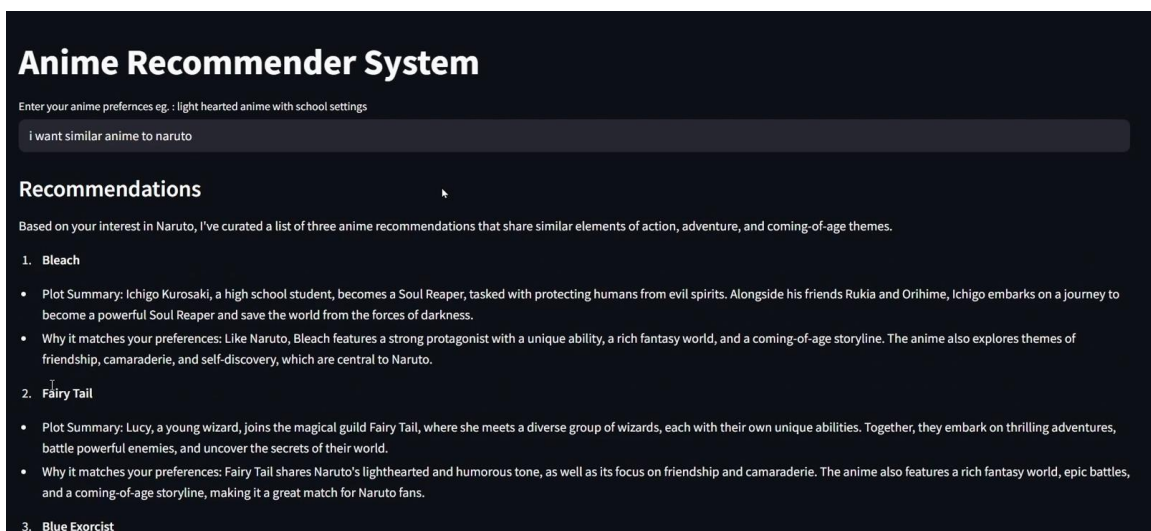
8. Key Steps / Modules

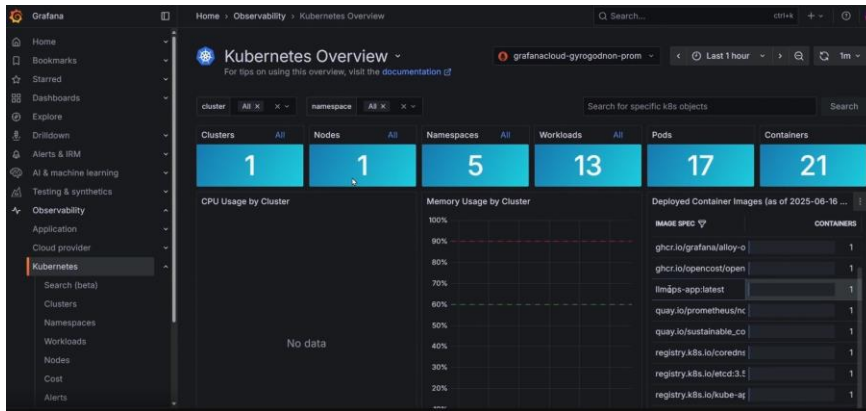
1. Dataset Collection and Preprocessing
2. Text Embedding Generation using HuggingFace Models
3. Vector Storage and Indexing using ChromaDB
4. User Query Input via Streamlit Interface
5. Semantic Similarity Search
6. Retrieval-Augmented Generation using LangChain
7. Conversational Response Generation
8. Deployment on Cloud with Monitoring

9. Learning Outcomes

- Understanding of Retrieval-Augmented Generation (RAG)
- Hands-on experience with vector databases and embeddings
- Practical knowledge of LangChain and LLM orchestration
- Building conversational AI applications using Streamlit
- Basics of containerization and orchestration
- Real-world cloud deployment and monitoring exposure

10. Screenshots





The screenshot shows the Grafana 'Workloads' dashboard for the 'grafanacloud-gyrogodnon-prom' cluster. It displays a table of workloads with their resource usage. The table has columns for 'WORKLOAD', 'TYPE', 'NAMESPACE', 'CLUSTER', and 'PODS'. The 'PODS' column includes a progress bar indicating the ratio of running pods to the total number of pods. The 'Resource usage' legend shows 'low' (green), 'med' (yellow), 'high' (red), and 'unknown' (grey).

WORKLOAD	TYPE	NAMESPACE	CLUSTER	PODS
grafana-k8s-monitoring-op...	deployment	monitoring	minikube	1 / 1
grafana-k8s-monitoring-win...	daemonset	monitoring	minikube	1 / 1
kube-apiserver-minikube	staticpod	kube-system	minikube	0 / 0
kube-controller-manager-mi...	staticpod	kube-system	minikube	0 / 1
kube-proxy	daemonset	kube-system	minikube	0 / 1
kube-scheduler-minikube	staticpod	kube-system	minikube	1 / 1
llmops-app	deployment	default	minikube	0 / 1
storage-provisioner	barepod	kube-system	minikube	1 / 1

11. Optional Add-ons

- User preference memory and personalization
- Anime posters and trailer integration
- Multi-language recommendation support
- User feedback-based recommendation improvement
- Public API for third-party integration

12. Resource Links

- HuggingFace: <https://huggingface.co>
- LangChain: <https://python.langchain.com>
- Docker: <https://docs.docker.com>
- Kubernetes: <https://kubernetes.io/docs>
- Minikube: <https://minikube.sigs.k8s.io/docs>
- kubectl: <https://kubernetes.io/docs/reference/kubectl>
- Google Cloud Platform: <https://cloud.google.com/docs>